

CHAPTER 4

FUNCTIONS

1.1. Functions

A function is a group of statements that together perform a task. Every C program has at least one function, which is `main()`, and all the most trivial programs can define additional functions.

You can divide up your code into separate functions. How you divide up your code among different functions is up to you, but logically the division is such that each function performs a specific task. A function can also be referred as a method or a sub-routine or a procedure, etc. A function declaration tells the compiler about a function's name, return type, and parameters. A function definition provides the actual body of the function.

There are basically two types of function those are

1. Library function
2. User defined function

The user defined functions defined by the user according to its requirement.

System defined function can't be modified; it can only read and can be used.

These function are supplied with every C compiler

Benefits of Functions

Novice programmers tend to pack all their code into `main()`, which soon becomes unmanageable. Scalable software design involves breaking a problem into sub-problems, which can each be tackled separately. Functions are the key to enabling such a division and separation of concerns.

- Functions allow a program to be split into a set of sub problems which, in turn, may be further split into smaller sub problems. This divide-and-conquer approach means that small parts of the program can be written, tested, and debugged in isolation without interfering with other parts of the program.
- Functions can wrap-up difficult algorithms in a simple and intuitive interface, hiding the implementation details, and enabling a higher-level view of the algorithm's purpose and use.

- Functions avoid code duplication. If a particular segment of code is required in several places, a function provides a tidy means for writing the code only once. This is of considerable benefit if the code segment is later altered.

Function Declarations

A function declaration tells the compiler about a function name and how to call the function. The actual body of the function can be defined separately.

A function declaration has the following parts:

Syntax

```
return_type function_name( parameter list );
```

Example:

```
int max(int num1, int num2);
```

Parameter names are not important in function declaration, only their type is required, so the following is also a valid declaration:

```
int max(int, int);
```

Function Definition

Function declaration is also known as function prototype. It informs the compiler about three things, those are name of the function, type of argument received by the function and the type of value returned by the function.

Syntax

```
datatype function_name(list of arguments with datatype)
{
    statements
    return value;
}
```

Before using any function it must be defined in the program. Function definition has three principal components: the first line, the parameter declarations and the body of the functions. The first line of a function definition contains the data type

of the information return by the function, followed by function name, and a set of arguments or parameters, separated by commas and enclosed in parentheses. The set of arguments may be skipped over. The data type can be omitted if the function returns an integer or a character. An empty pair of parentheses must follow the function name if the function definition does not include any argument or parameters.

Actual and formal arguments

Passing of values between the main program and the function takes place through arguments.

The arguments listed in the function calling statements are referred to as actual arguments. These actual values are passed to a function to compute a value or to perform a task. The arguments used in the function declaration are referred as formal arguments. They are simply formal variables that accept or receive the values supplied by the calling function. The number of actual and formal arguments and their data types should match.

Accessing a function

A function is accessed in the program (known as calling program) by specifying its name with optional list of arguments enclosed in parenthesis. If arguments are not required then use empty parenthesis.